Collections

The Java Collections Library

A collection is a group of objects organized in some way.

The Java Collections Library is a set of reusable components:

Interfaces

- `Collection<T>, List<T>, Set<T>, Map<K, V>…`

Implementations

- `ArrayList<T>, LinkedList<T>, HashSet<T>, HashMap<K, V> …`

Algorithms

- Finding the minimum or maximum element in a collection.
- Sorting a collection.
- Shuffling a collection.
- …

Examples of Collections

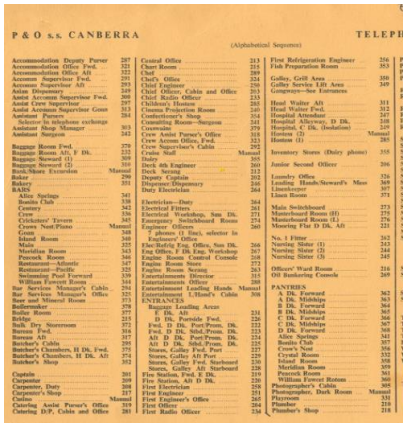
A poker hand (a collection of cards)

A mailbox (a collection of letters)

A telephone directory (a mapping from names to numbers)







Why use Java Collections?

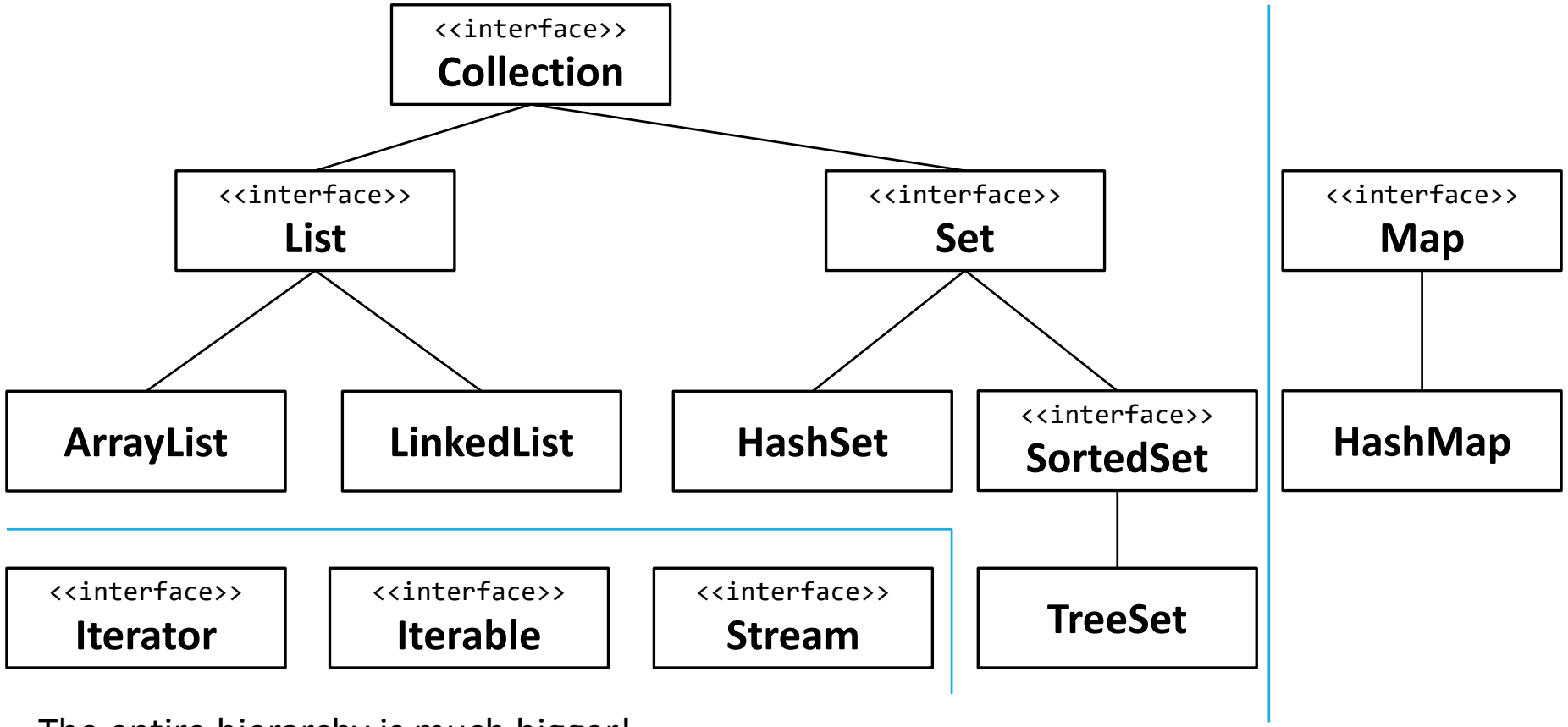
Why should you prefer collections over ordinary arrays?

- Arrays are a low-level abstraction, essentially a chunk of memory.
- Arrays offer few operations and are cumbersome.
- Collections are a high-level abstraction.
- Collections offer rich operations and are easy to use.
- We can save time. We don't have to re-invent the wheel.
- Everyone agrees to use collections. Better for reuse.
- The collections are heavily optimized. Probably better than we could do.

⇒ **Arrays be gone! Use collections from now on!**

(Unless you have good reason not to)

The Java Collections Hierarchy



The entire hierarchy is much bigger!

Overview

`Collection<E>` Defines common methods for all collections.

`List<E>` A sequence of ordered elements.

- The user controls the order of the elements.
- Defines methods to insert and retrieve elements from specific positions.

`Set<E>` An unordered collection of elements without duplicates.

- Relies on the `equals` method to eliminate duplicates.

SortedSet<E> A sorted collection of elements without duplicates.

- Relies on the *natural ordering* of elements or on a comparator.

`Map<K, V>` A mapping between keys of type K and values of type V.

- Relies on the `equals` method to compare keys.

Many other important collections: Stack<E>, Queue<E>, Deque<E>, ...

Collection<E>

Return Type	Method
`boolean`	**`add(E e)`** Ensures that this collection contains the specified element (optional operation).
`void`	**`clear()`** Removes all of the elements from this collection (optional operation).
`boolean`	**`contains(Object o)`** Returns true if this collection contains the specified element.
`boolean`	**`isEmpty()`** Returns true if this collection contains no elements.
`Iterator<E>`	**`iterator()`** Returns an iterator over the elements in this collection.
`boolean`	**`remove(Object o)`** Removes a single instance of the specified element from this collection, if it is present (optional operation).
`int`	**`size()`** Returns the number of elements in this collection.
`Stream<E>`	**`stream()`** Returns a sequential Stream with this collection as its source.

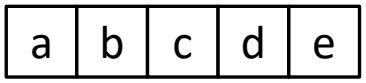
List<E>

Return Type	Method
`void`	**`add(int index, E element)`** Inserts the specified element at the specified position in this list (optional operation).
`E`	**`get(int index)`** Returns the element at the specified position in this list.
`int`	**`indexOf(Object o)`** Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
`int`	**`lastIndexOf(Object o)`** Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
`E`	**`remove(int index)`** Removes the element at the specified position in this list (optional operation).
`List<E>`	**`subList(int fromIndex, int toIndex)`** Returns a view of the portion of this list between the specified fromIndex, inclusive, and toIndex, exclusive.

ArrayList<E> and LinkedList<E>

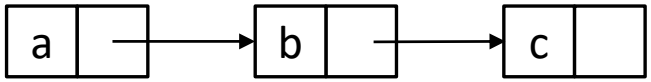
`ArrayList<E>` A list implementation backed by an array.

- Good random access performance.
  - Quick to find an element given an index.
- Cannot insert elements in the front.



`LinkedList<E>` A list implementation backed by linked cells.

- Bad random access performance.
  - Slow to find an element given an index, must scan through the list.
- Can insert elements in the front.



Most common to use `ArrayList<E>`.

Example: List<E>

```
public static void main(String[] args) {

    List<String> cards = new ArrayList<>();
    cards.add("Ace of Hearts");
    cards.add("Queen of Spades");
    cards.add("Ten of Hearts");
    cards.add("Ace of Hearts");
    cards.add("Two of Clubs");

    cards.remove(1);
    int index = cards.indexOf("Ten of Hearts");
    System.out.println(index);

}
```

Collections

Return Type	Method
`static int`	**`binarySearch(List<T> list, T key)`** Searches the specified list for the specified object using the binary search algorithm.
`static int`	**`binarySearch(List<T> list, T key, Comparator<T> c)`** Searches the specified list for the specified object using the binary search algorithm.
`static void`	**`shuffle(List<T> list)`** Randomly permutes the specified list using a default source of randomness.
`static void`	**`sort(List<T> list)`** Sorts the specified list into ascending order, according to the natural ordering of its elements.
`static void`	**`sort(List<T> list, Comparator<T> c)`** Sorts the specified list according to the order induced by the specified comparator.

(Note: I simplified some of the types for readability.)

Comparator<T>

```
public interface Comparator<T> {

    /**
     * Compares its two arguments for order.
     *
     * Returns a negative integer, zero, or a positive integer
     * as the first argument is less than, equal to, or greater
     * than the second.
     */
    int compare(T o1, T o2);
}
```

Example: Set<E>

```
public static void main(String[] args) {

    Set<String> cards = new HashSet<>();
    cards.add("Ace of Hearts");
    cards.add("Ace of Hearts");
    cards.add("Queen of Spades");
    cards.add("Queen of Spades");
    cards.add("Ten of Hearts");
    cards.add("Two of Clubs");

    cards.remove("Ten of Hearts");
    int size = cards.size();
    System.out.println(size);

}
```

Map<K, V>

Return Type	Method
boolean	**`containsKey(Object key)`** Returns true if this map contains a mapping for the specified key.
boolean	**`containsValue(Object value)`** Returns true if this map maps one or more keys to the specified value.
V	**`get(Object key)`** Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
V	**`getOrDefault(Object key, V defaultValue)`** Returns the value to which the specified key is mapped, or defaultValue if this map contains no mapping for the key.
Set<K>	**`keySet()`** Returns a Set view of the keys contained in this map.
V	**`put(K key, V value)`** Associates the specified value with the specified key in this map (optional operation).
Collection<V>	**`values()`** Returns a Collection view of the values contained in this map.

Example: Map

```
public static void main(String[] args) {

    Map<String, Integer> birthYear = new HashMap<>();
    birthYear.put("George Washington", 1732);
    birthYear.put("Abraham Lincoln", 1809);
    birthYear.put("Barack Obama", 1961);

    Integer obamaYear = birthYear.get("Barack Obama");
    System.out.println("Mr. Obama was born in: " + obamaYear);

    System.out.println("We have information on the presidents:");
    for (String president : birthYear.keySet()) {
        System.out.println("Mr. " + president);
    }

}
```